

Recursion Pattern Learning Notes By Aman

Mastering Recursion: A Complete Guide to Recursive Patterns

Recursion is often intimidating because it requires a shift from "how to do it" (loops) to "what it is" (definitions). To master recursion, you don't just learn code; you learn to recognize **patterns**.

This document breaks down the core patterns of recursion used in software engineering and competitive programming.

1. Linear Recursion

This is the most basic form where a function makes exactly **one** recursive call at each step.

How to Identify It

- You are processing a list, string, or a number one step at a time.
- The problem can be solved by doing something to the current element and asking the function to handle the "rest."

The Thought Process

1. **Base Case:** What is the smallest possible input? (e.g., $n = 0$ or an empty string).
2. **Recursive Step:** If I solve just the current unit, how do I combine it with the result of the remaining units?

Java



```
public void linear(int n) {  
    if (n == 0) return; // Base Case  
    // Do something  
    linear(n - 1); // Single recursive call  
}
```

Example: Factorial

- **Why:** Each number depends only on the number before it.
- **Code:** `return n * factorial(n - 1);`

2. Divide and Conquer Recursion

In this pattern, the problem is split into **two or more independent sub-problems**, usually of equal size.

How to Identify It

- The input can be halved (like an array or a range of numbers).
- The sub-problems do not overlap (they work on different parts of the data).

The Thought Process

1. **Divide:** Split the problem into two halves.
2. **Conquer:** Call the function on both halves.
3. **Combine:** Merge the results from both halves to get the final answer.

Java

```
public int divideAndConquer(int[] arr, int left, int right) {  
    if (left == right) return arr[left]; // Base Case  
  
    int mid = left + (right - left) / 2;  
    int leftResult = divideAndConquer(arr, left, mid);  
    int rightResult = divideAndConquer(arr, mid + 1, right);  
  
    return merge(leftResult, rightResult); // Combine  
}
```

Example: Merge Sort or Binary Search

3. Multiple Recursion (The Fibonacci Pattern)

This is where a function calls itself multiple times, but unlike Divide and Conquer, the sub-problems often **overlap**.

How to Identify It

- The current state depends on several previous states (e.g., $f(n) = f(n - 1) + f(n - 2)$).
- The growth of calls is exponential.

Why use it?

It is the most direct way to translate a mathematical recurrence relation into code. Note that for efficiency, this pattern usually requires **Memoization** to avoid redundant work.

4. Subsets / Subsequences Pattern

This is the "Pick or Don't Pick" pattern. It is the foundation of many dynamic programming and backtracking problems.

How to Identify It

- You need to find all possible combinations of elements.
- The order doesn't necessarily matter, but the inclusion does.

The Thought Process

At every element, you have a binary choice:

1. **Include** the current element in your collection.
2. **Exclude** the current element from your collection.

Code Template

Java

```
public void findSubsets(int index, List<Integer> current, int[] nums) {
    if (index == nums.length) {
        System.out.println(current);
        return;
    }

    // Pick the element
    current.add(nums[index]);
    findSubsets(index + 1, current, nums);

    // Backtrack: Don't pick the element
    current.remove(current.size() - 1);
    findSubsets(index + 1, current, nums);
}
```

5. Permutation Pattern

Unlike subsets, permutations care about **order**.

How to Identify It

- You need to rearrange all given elements in every possible way.
- The input size stays the same; only the sequence changes.

The Thought Process

1. You have a "pool" of available characters.
2. Loop through the pool, pick one to be at the current position, and recurse for the remaining characters.

```
public void permutations(String processed, String unprocessed)
{
    if (unprocessed.isEmpty())
    {
        System.out.println(processed);
        return;
    }

    for (int i = 0; i < unprocessed.length(); i++)
    {
        char ch = unprocessed.charAt(i);
        // New unprocessed is everything except the char at i
        String newUnprocessed = unprocessed.substring(0, i) + unprocessed.substring(i + 1);
        permutations(processed + ch, newUnprocessed);
    }
}
```

6. Combination Sum Pattern

This pattern is used when you need to find sets of numbers that add up to a specific target, often allowing for the reuse of elements.

The Thought Process

- **Target logic:** Instead of keeping track of a sum, pass `target - current_element`.
- **Base Case:** If `target == 0`, you found a valid combination. If `target < 0`, stop.

Now Detail Pattern Recognition

Pattern 1: Linear Recursion

Linear Recursion is the most fundamental building block of recursive thinking. It is called "linear" because the function makes exactly **one** recursive call at each step, forming a straight line of execution until it hits a base case.

1. How to Identify It

You are likely looking at a linear recursion problem if:

- The problem involves a sequence (an array, a string, or a range of numbers).
- The total work can be defined as: **"Do one small thing now, and let the function handle the rest of the sequence."**
- The state changes by a constant amount (usually $n - 1$) in every step.

2. The Mental Model: "The Delegate"

To think in linear recursion, stop trying to visualize every single step. Instead, follow this **Three-Step Thought Process**:

1. **The Base Case (The Stopping Point):** What is the simplest possible version of this problem that I can answer instantly?
 - *Example:* If summing an array, the base case is an empty array (Sum = 0).
2. **The Recursive Relation (The "Self-Similarity"):** How does the answer for N relate to the answer for $N - 1$?
 - *Logic:* "I will handle the first element. I will trust the function to give me the correct sum of the remaining $N - 1$ elements."
3. **The Combination:** How do I combine my "small work" with the "delegated result"?
 - *Logic:* `Result = MyCurrentElement + SumOfRemainingElements`.

3. The Anatomy of Linear Recursion

There are two ways this pattern executes, depending on *when* you do the work:

- **Head Recursion:** You do the work **after** the recursive call returns (climbing back up the stack).
- **Tail Recursion:** You do the work **before** the recursive call (passing the result down).

4. Detailed Example: Factorial of N ($N!$)

The mathematical definition is $n! = n \times (n - 1) \times (n - 2) \cdots \times 1$.

Step-by-Step Thought Process:

- **Identify:** To find $5!$, I need $4!$. To find $4!$, I need $3!$. This is a sequence decreasing by 1.
- **Base Case:** If $n = 1$ or $n = 0$, the answer is 1.
- **Recursive Step:** $Factorial(n) = n \times Factorial(n - 1)$.

Pattern 2: Multiple Recursion (The Fibonacci / Tree Pattern)

While Linear Recursion makes **one** call, **Multiple Recursion** makes **two or more** recursive calls within the same function. This is how we move from a "line" of execution to a "tree."

1. How to Identify It

You are likely dealing with Multiple Recursion if:

- The current state depends on **multiple previous states** (e.g., $f(n)$ depends on both $f(n - 1)$ and $f(n - 2)$).
- You are exploring **multiple paths** or branches (e.g., a decision tree).
- The problem structure resembles a **Hierarchy** or a **Tree**.

2. The Thought Process: "The Splitter"

In Linear recursion, you think: "*I do one part, you do the rest.*" In Multiple recursion, you think: "*I am the root, and I need results from my left branch and my right branch to give my answer.*"

1. **The Base Case:** Usually requires handling the two smallest inputs (e.g., for Fibonacci, you need to define both $n = 0$ and $n = 1$).
2. **The Recursive Relation:** $Result = Function(PartA) + Function(PartB)$.
3. **Visualization:** Every call branches out, doubling the work at every level. This leads to an **Exponential Time Complexity** ($O(2^n)$) unless optimized.

The sequence is: 0, 1, 1, 2, 3, 5, 8, 13 . . . The rule is: Each number is the sum of the two preceding ones.

Java Implementation

Java



```
public class MultipleRecursion {
    public static int fibonacci(int n) {
        // 1. Base Cases: We need two because we look back two steps
        if (n == 0) return 0;
        if (n == 1) return 1;

        // 2. Multiple Recursive Calls
        // The function pauses here, calculates left, then calculates right
        int leftBranch = fibonacci(n - 1);
        int rightBranch = fibonacci(n - 2);

        // 3. Combination
        return leftBranch + rightBranch;
    }

    public static void main(String[] args) {
        System.out.println(fibonacci(6)); // Output: 8
    }
}
```

Pattern 3: Divide and Conquer (D&C)

Divide and Conquer is often confused with Multiple Recursion. While it *does* make multiple calls, the key difference is in the **Input Management**. In Multiple Recursion, the sub-problems often overlap (like Fibonacci). In Divide and Conquer, you split the data into **disjoint (non-overlapping)** halves.

1. How to Identify It

- You are dealing with large datasets (Arrays, Lists, or Ranges).
- The problem can be solved by breaking the data into two equal (or near-equal) halves.
- The "Combine" step is usually where the magic happens (e.g., merging sorted lists).

2. The Thought Process: "The Efficiency Expert"

The goal here is to reduce the "depth" of the recursion. By halving the input every time, we reach the base case in $\log n$ steps instead of n steps.

1. **Divide:** Find the middle point (`mid`) and split the range.
2. **Conquer:** Recursively solve the left half and the right half.
3. **Combine:** Take the two solved halves and merge them into a single solution.

3. Detailed Example: Finding the Maximum Element in an Array

While you can find the Max using Linear Recursion, using D&C is a great way to understand the structure.

Java



```
public class DivideAndConquer {
    public static int findMax(int[] arr, int start, int end) {
        // 1. Base Case: Only one element left
        if (start == end) {
            return arr[start];
        }

        // 2. Divide
        int mid = start + (end - start) / 2;

        // 3. Conquer (Recursive calls on halves)
        int leftMax = findMax(arr, start, mid);
        int rightMax = findMax(arr, mid + 1, end);

        // 4. Combine (The actual logic)
        return Math.max(leftMax, rightMax);
    }

    public static void main(String[] args) {
        int[] nums = {3, 8, 1, 9, 4, 2, 7, 5};
        System.out.println("Max element: " + findMax(nums, 0, nums.length - 1))
    }
}
```

4. Why use this pattern?

- **Performance:** This is the logic behind $O(n \log n)$ algorithms like **Merge Sort** and **Quick Sort**.
- **Parallelism:** Since the left and right halves are independent, you could technically run them on different CPU cores simultaneously.
- **Scalability:** It works efficiently on massive amounts of data.

5. Code Template for Divide and Conquer

Java

```
public ReturnType solve(int[] data, int start, int end) {  
    // 1. Base Case: Smallest possible range  
    if (start >= end) {  
        return handleSmallest(data[start]);  
    }  
  
    // 2. Divide: Split the range in two  
    int mid = start + (end - start) / 2;  
  
    // 3. Conquer: Recursive calls  
    ReturnType leftResult = solve(data, start, mid);  
    ReturnType rightResult = solve(data, mid + 1, end);  
  
    // 4. Combine: Merge the results  
    return merge(leftResult, rightResult);  
}
```

Pattern 4: The Subset Pattern ("Pick or Don't Pick")

This is arguably the most important pattern for coding interviews and competitive programming. It is the foundation of **Backtracking** and **Dynamic Programming**.

1. How to Identify It

- The problem asks for **all possible combinations, subsequences, or subsets**.
 - The order of elements doesn't necessarily change, but whether an element is "in" or "out" does.
 - Keywords: "Find all subsets," "Power set," "Combination sum," "Target sum."
-

2. The Thought Process: "The Binary Decision"

In this pattern, you don't look at the whole array at once. You stand at a specific index and ask a single question: **"Does this element belong in my current collection?"**

1. **Choice A (Pick):** Add the element to your "current list" and move to the next index.
2. **Choice B (Don't Pick):** Leave the "current list" as it is and move to the next index.

By making these two choices at every step, you naturally explore every possible combination of elements.

3. Detailed Example: Generating all Subsets

If you have `[1, 2]`, the subsets are `[]`, `[1]`, `[2]`, and `[1, 2]`.

Java Implementation

Java



```
import java.util.*;

public class SubsetPattern {
    public static void findSubsets(int index, int[] nums, List<Integer> current)
    {
        // 1. Base Case
        // If we have reached the end of the array, we have formed one subset
        if (index == nums.length) {
            System.out.println(current);
            return;
        }

        // 2. PICK the element at the current index
        current.add(nums[index]);
        findSubsets(index + 1, nums, current);

        // 3. DON'T PICK the element (Backtrack)
        // We remove the last element to "undo" the pick and explore the other path
        current.remove(current.size() - 1);
        findSubsets(index + 1, nums, current);
    }

    public static void main(String[] args) {
        int[] nums = {1, 2, 3};
        findSubsets(0, nums, new ArrayList<>());
    }
}
```

4. Why use this pattern?

- **Completeness:** It guarantees you find every single mathematical subset (2^n results).
- **State Management:** It teaches you how to "clean up" your state (the `remove` step), which is essential for more complex problems like Sudoku solvers or N-Queens.
- **Filtering:** You can easily add logic to the base case (e.g., "only print if the sum of elements equals 10").

5. Code Template for Subsets

Java



```
public void solve(int index, List<Type> currentPath, InputType[] inputs) {  
    // Base Case: We've made a decision for every input  
    if (index == inputs.length) {  
        process(currentPath);  
        return;  
    }  
  
    // Step 1: Include current element  
    currentPath.add(inputs[index]);  
    solve(index + 1, currentPath, inputs);  
  
    // Step 2: Exclude current element (Backtracking step)  
    currentPath.remove(currentPath.size() - 1);  
    solve(index + 1, currentPath, inputs);  
}
```

6. The "Backtracking" Concept

Notice the `current.remove(...)` line. This is crucial. Because we are passing the same `current` list object through all recursive calls, we must **undo** our choice before we return to the previous level of the tree. If we didn't remove it, the "Don't Pick" branch would accidentally contain the element we picked in the previous branch!

Pattern 5: Permutations & Combinations

1. The Combination Pattern (Selection)

Combinations are essentially a variation of the **Subset Pattern** we just discussed.

- **Goal:** Choose k elements from a set of n .
- **Key Rule:** Order **does not** matter ($[1, 2]$ is the same as $[2, 1]$).
- **Identification:** "Find all groups of size 3," "Pick 2 items from the menu."

The Thought Process: "The Boundary Marker"

To avoid duplicates like $[1, 2]$ and $[2, 1]$, we use a `start` index. Once we pick an element, we **never look back** at the elements before it.

To avoid duplicates like $[1, 2]$ and $[2, 1]$, we use a `start` index. Once we pick an element, we **never look back** at the elements before it.

Java Implementation (nCr)

Java



```
public void combinations(int start, int[] nums, int k, List<Integer> current) {  
    // Base Case: If the current list size is k, we found a combination  
    if (current.size() == k) {  
        System.out.println(current);  
        return;  
    }  
  
    for (int i = start; i < nums.length; i++) {  
        current.add(nums[i]);           // Pick element  
        combinations(i + 1, nums, k, current); // Recurse with i+1 (don't look back)  
        current.remove(current.size() - 1); // Backtrack  
    }  
}
```

2. The Permutation Pattern (Arrangement)

Permutations are about rearranging elements.

- **Goal:** Find all possible orders of a set.
- **Key Rule:** Order matters ($[1, 2]$ is different from $[2, 1]$).
- **Identification:** "Find all anagrams," "Arrange people in a line," "Generate all routes."

The Thought Process: "The Occupied Seat"

In Permutations, every element wants a turn at every position.

1. **Level 1:** Who sits in the first chair? (Any of the n elements).
2. **Level 2:** Who sits in the second chair? (Any of the *remaining* $n - 1$ elements).

There are two common ways to code this. The **Swapping Method** is the most memory-efficient.

Java



```
public void permutations(int[] nums, int index) {
    // Base Case: If the index reaches the end, one arrangement is complete
    if (index == nums.length) {
        System.out.println(Arrays.toString(nums));
        return;
    }

    for (int i = index; i < nums.length; i++) {
        swap(nums, index, i);           // Swap current index with i
        permutations(nums, index + 1);  // Recurse for the next position
        swap(nums, index, i);           // Backtrack (Swap back) to restore array
    }
}

private void swap(int[] nums, int i, int j) {
    int temp = nums[i];
    nums[i] = nums[j];
    nums[j] = temp;
}
```

